Prime Numbers

Kanishk Goel

16 May 2026

1 Sieve of Eratosthenes

Used to find all prime numbers in a segment $[1 : n]$

Idea: Mark all proper multiples of 2 as composite, find the next non composite number (3 in this case), mark all its proper multiples as composite and so on.

Runtime: $O(n\ log(log(n)))$ and space complexity is $O(n)$.

1.1 Prime Number Facts

- number of prime numbers less than or equal to n are $\frac{n}{logn}$
- k^{th} prime number approximately equals $k\,ln(k)$.

1.2 Optimizations

- sieving till root.
- sieving by odd numbers only.
- starting the sieve check for a prime number p from $p \cdot p$ because any multiple of p less than that has been marked by a prime number smaller than p.
- Better to use vector <bool > instead of vector <char> because it takes lesser memory hence is able to utilise cache better. (<bool> is N/8 bytes while char is N bytes (bits vs bytes) Although architecture utilises bytes better, cache optimization in less memory load is significantly more helpful.)

2 Segmented Sieve

Instead of storing the entire array `is_prime`$[1 \dots n]$, it is sufficient to store only the primes up to $\sqrt{n}$. The range $[1, n]$ is divided into blocks of size S.
The k-th block contains:

$$[kS,\ kS + S - 1]$$

for

$$k = 0, 1, \dots, \left\lfloor \frac{n}{S} \right\rfloor .$$

For each block:

- Iterate through all primes $\leq \sqrt{n}$.
- Mark their multiples within the current block as composite.

Special care is needed for:

- Preventing primes $\leq \sqrt{n}$ from marking themselves.
- Marking 0 and 1 as non-prime.
- n is not necessarily at the end of the last block.

Time Complexity: $O(n \log \log n)$ and Memory Usage: $O(\sqrt{n} + S)$.

This approach also improves cache efficiency. However, very small block sizes increase division overhead, so practical block sizes are usually chosen between 10^4 and 10^5.

3 Linear Sieve

Maintains an array lp, $lp[i]$ denotes minimum prime factor of i. $lp[i] = i$ for primes i eventually. Therefore, requires $O(n)$ memory space. It should thus be used for not more than $n = 10^7$. Initially, $lp[i] = 0$, $\forall i$.

Algorithm

```
lp[i] ← 0 ∀ i
pr ← empty vector
foreach 2 ≤ i ≤ N, i+ = 1 do
    if lp[i] = 0 then
        lp[i] ← i
        Push i into pr
foreach j ≥ 0 such that i × pr[j] ≤ n, i+ = 1 do
    lp[i × pr[j]] ← pr[j]
    if pr[j] == lp[i] then
        break
```

Practically this algorithm runs as fast as normal sieve of eratosthenes, and slower than optimized versions such as segmented sieve. But, it can be used to find factorization of numbres, which is extremely uesful.